

OPENCV MASTER CHEAT SHEET

1. Core I/O & Basic Properties

```
import cv2
import numpy as np

# Read & Save Images
img = cv2.imread("image.jpg")
cv2.imwrite("new.jpg", img)

# Display Images
cv2.imshow("Window", img)
cv2.waitKey(0) # 0 = Wait indefinitely
cv2.destroyAllWindows()

# Properties
h, w, c = img.shape # Height, Width, Channels
```

2. Processing & Colors

```
# Color Conversions (BGR is default)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Split Color Channels (B, G, R)
b = img[:, :, 0]
g = img[:, :, 1]
r = img[:, :, 2]

# Blurring & Smoothing
blur = cv2.GaussianBlur(img, (7,7), 0)

# Thresholding (Grayscale to Binary)
_, mask = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

3. Geometric Transformations

```
# Cropping (NumPy Slicing: [y, x])
crop = img[0:300, 0:500] # [y1:y2, x1:x2]

# Resizing (width, height)
resized = cv2.resize(img, (300, 300))

# Flipping
# 0 = Vertical, 1 = Horizontal, -1 = Both
flipped = cv2.flip(img, 1)

# Rotation
center = (w // 2, h // 2)
matrix = cv2.getRotationMatrix2D(center, 90, 1.0)
rotated = cv2.warpAffine(img, matrix, (w, h))
```

4. Drawing & Annotations

```
# Blank Canvas (Black)
blank = np.zeros((512, 512, 3), dtype=np.uint8)

# Shapes (img, point1, point2, color_BGR, thickness)
# thickness = -1 means filled
cv2.rectangle(img, (100,100), (300,300), (255,0,0), -1)
cv2.circle(img, (250,250), 50, (0,0,255), 3)
cv2.line(img, (0,0), (512,512), (0,255,0), 2)

# Text
cv2.putText(img, "Hello", (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)
```

5. Video & Webcam

```
# 0 = Default webcam, or use "video.mp4"
cap = cv2.VideoCapture(0)

# Setup Video Writer to Save
fourcc = cv2.VideoWriter_fourcc(*'XVID')
out = cv2.VideoWriter("out.avi", fourcc, 20.0, (640, 480))

while True:
    ret, frame = cap.read()
    if not ret: break

    out.write(frame)
    cv2.imshow("Feed", frame)

    # Quit on 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

6. Advanced: Face & YOLO

```
# Edge Detection
edges = cv2.Canny(gray, 100, 200)

# Face Detection (Haar Cascades)
face_cascade = cv2.CascadeClassifier("haarcascade.xml")
faces = face_cascade.detectMultiScale(gray, 1.1, 5)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x,y), (x+w, y+h), (0,255,0), 2)

# YOLOv8 Object Detection (Ultralytics)
from ultralytics import YOLO
model = YOLO("yolov8n.pt") # Load Nano model

results = model(frame)
annotated_frame = results[0].plot()
cv2.imshow("YOLO", annotated_frame)
```